

Item-restscore and infit under misfit

Preliminary power and flag-direction study, 7 polytomous items

Magnus Johansson

2026-10-01

Table of contents

Purpose	1
Design	1
Data generation	3
One dataset	3
Run	4
Results	5
Detection of the misfitting items	5
False flags on the fitting items, by direction	8
Clean decisions	9
Paired comparison of restscore and infit	11
Failed datasets	11
Interpretation	12

Purpose

Two observations from practice, comparing conditional infit with simulation-based cutoffs against item-restscore with BH correction:

1. Underfitting items are flagged later by item-restscore than by infit.
2. Overfitting items are flagged more often by item-restscore than by infit.

`restscore_asymptotic_null.qmd` explains part of both under a true Rasch model. The asymptotic item-restscore test rejects too rarely in the underfit tail and, at small n , too often in the overfit tail, and 78 to 100 percent of its BH false flags are overfit. That leaves open what happens when items genuinely misfit. The overfit flags seen in practice could come from three places:

- **Null miscalibration**, which the bootstrap null in `RMitemRestscoreCutoff()` is designed to remove.
- **The cascade**. When one item underfits, the others look relatively over-discriminating, and the reverse.
- **Sensitivity**. Gamma may respond more strongly than infit to genuine over-discrimination.

This study plants items that under- or over-discriminate and records who flags what, and in which direction. Running the asymptotic and the bootstrap item-restscore tests on the same datasets separates the first explanation from the other two.

Design

- **Scale**. 7 polytomous items, 5 response categories, thresholds at the item location ± 0.5 and ± 1.5 . $\theta \sim N(0, 1.5^2)$, as in the null studies.
- **Underfit** is generated by multidimensionality, as in Johansson (2025) and the FWER paper: the misfitting items respond to a second dimension correlated $\rho = .85$ with the first. With two misfitting items they share that dimension. The shared residual makes them locally dependent on each other, which the FWER paper found makes them mask each other. It may also raise their association with the rest score, which bears directly on how item-restscore labels them.

- **Overfit** is generated by a discrimination above 1, with slope $a = 1.6$ under the generalised partial credit model. All other items have $a = 1$.
- **Calibration.** Both values are anchored on the infit scale, as in the FWER paper, against the phq9 items flagged at $n = 600$: underfit 1.23 to 1.31 (q3, q8, q9) and overfit 0.78 to 0.83 (q2, q4). `restscore_power_calibration.R` scanned one misfitting item at $n = 600$ (80 replications per value). $\rho = .85$ gave infit 1.32 on target and 1.27 at -2 logits. $a = 1.6$ gave 0.79 and 0.80. The parameter is not portable across scale lengths (the FWER paper needed $\rho = .75$ at five items), so it was scanned for this scale rather than carried over.
- **Number of misfitting items.** 1 (item I1) or 2 (I1 and I2).
- **Targeting of the misfitting item(s).** Location 0 (the person mean) or -2 . With two items they sit at the target ± 0.25 . The fitting items are evenly spread over $[-1.5, 1.5]$.
- **Sample size.** 150, 250, 400, 600.
- **Replications.** 100 datasets per cell, 32 cells. The Monte Carlo SE of a detection rate is at most 5 points. That is enough for preliminary results, not for fine distinctions.

Methods, all at $\alpha = .05$, on the same dataset:

label	method
<code>infit_wy</code>	<code>RMitemInfit()</code> with <code>RMitemInfitCutoff()</code> , Westfall-Young (package default)
<code>infit_bh</code>	the same cutoff object, <code>correction = "fdr_bh"</code>
<code>rest_boot_wy</code>	<code>RMitemRestscore()</code> with <code>RMitemRestscoreCutoff()</code> , Westfall-Young
<code>rest_boot_bh</code>	the same cutoff object, <code>correction = "fdr_bh"</code>
<code>rest_asy_bh</code>	<code>RMitemRestscore()</code> without a cutoff, asymptotic BH (package default)

Both cutoff functions use the package defaults: 400 iterations and the resample DGP.

Outcomes.

- **Detection:** a misfitting item flagged in the correct direction.
- **Wrong direction:** a misfitting item flagged in the opposite direction.
- **False flags on fitting items**, split into overfit and underfit. This is where the cascade shows.
- **Clean decision:** every misfitting item correctly flagged and no fitting item flagged.

Cost. About 0.1 s per bootstrap iteration for the two cutoff functions together, so 40 s per dataset. 3,200 datasets is about 36 CPU-hours, or 3.5 to 4 hours on 10 cores. Results are saved per cell, so an interrupted render resumes.

```
suppressMessages({
  devtools::load_all("...", quiet = TRUE)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})
options(rgl.useNULL = TRUE)

R_DATASETS <- 100
B           <- 400
RHO        <- 0.85 # underfit, from restscore_power_calibration.R
SLOPE      <- 1.6  # overfit, from restscore_power_calibration.R
NCORES     <- 10
```

```

RES_DIR    <- "restscore_infit_power"
dir.create(RES_DIR, showWarnings = FALSE)

cells <- expand.grid(
  n = c(150, 250, 400, 600),
  target = c(0, -2),
  n_misfit = c(1, 2),
  direction = c("underfit", "overfit"),
  stringsAsFactors = FALSE
)
cells$cell <- with(cells, sprintf("%s_%ditem_t%s_n%d", direction, n_misfit,
                                ifelse(target == 0, "0", "m2"), n))
nrow(cells)

```

```
[1] 32
```

Data generation

The generator is shared with the calibration script, so the calibrated values mean the same thing in both.

```

source("restscore_power_generator.R")
item_setup(2, -2)

```

```

$thr
$thr[[1]]
[1] -3.75 -2.75 -1.75 -0.75

$thr[[2]]
[1] -3.25 -2.25 -1.25 -0.25

$thr[[3]]
[1] -3 -2 -1 0

$thr[[4]]
[1] -2.25 -1.25 -0.25 0.75

$thr[[5]]
[1] -1.5 -0.5 0.5 1.5

$thr[[6]]
[1] -0.75 0.25 1.25 2.25

$thr[[7]]
[1] 0 1 2 3

$misfit
[1] 1 2

```

One dataset

A dataset with an unobserved category is redrawn, as in the validation study.

```

one_dataset <- function(rep, cell_row) {
  setup <- item_setup(cell_row$n_misfit, cell_row$target)
  seed <- 1e6 * match(cell_row$cell, cells$cell) + rep
  d <- sim_power_data(
    n = cell_row$n, n_misfit = cell_row$n_misfit, target = cell_row$target,
    direction = cell_row$direction, rho = RH0, slope = SLOPE, seed = seed
  )

  tryCatch({
    ci <- suppressMessages(RMitemInfitCutoff(
      d, iterations = B, parallel = FALSE, seed = seed,
      cutoff_method = "quantile"
    ))
    cr <- suppressMessages(RMitemRestscoreCutoff(
      d, iterations = B, parallel = FALSE, seed = seed,
      cutoff_method = "quantile"
    ))
    flags <- function(tab) tab$Flagged[match(names(d), tab$Item)]
    q <- function(expr) suppressWarnings(suppressMessages(expr))
    data.frame(
      rep = rep,
      item = names(d),
      misfit = seq_len(7) %in% setup$misfit,
      infit_wy = flags(q(RMitemInfit(d, cutoff = ci, output = "dataframe"))),
      infit_bh = flags(q(RMitemInfit(d, cutoff = ci, correction = "fdr_bh",
        output = "dataframe"))),
      rest_boot_wy = flags(q(RMitemRestscore(d, cutoff = cr,
        output = "dataframe"))),
      rest_boot_bh = flags(q(RMitemRestscore(d, cutoff = cr,
        correction = "fdr_bh",
        output = "dataframe"))),
      rest_asy_bh = flags(q(RMitemRestscore(d, output = "dataframe"))),
      stringsAsFactors = FALSE
    )
  }, error = function(e) NULL)
}

```

Run

```

for (i in seq_len(nrow(cells))) {
  f <- file.path(RES_DIR, paste0(cells$cell[i], ".rds"))
  if (file.exists(f)) next
  t0 <- Sys.time()
  res <- mclapply(seq_len(R_DATASETS), function(r) one_dataset(r, cells[i, ]),
    mc.cores = NCORES, mc.preschedule = FALSE)
  saveRDS(list(res = res,
    elapsed_min = as.numeric(Sys.time() - t0, units = "mins")),
    f)
}

methods <- c("infit_wy", "infit_bh", "rest_boot_wy", "rest_boot_bh",
  "rest_asy_bh")

long <- do.call(rbind, lapply(seq_len(nrow(cells)), function(i) {

```

```

x <- readRDS(file.path(RES_DIR, paste0(cells$cell[i], ".rds")))
ok <- Filter(Negate(is.null), x$res)
cbind(cells[i, c("direction", "n_misfit", "target", "n")],
      do.call(rbind, ok), failed = length(x$res) - length(ok),
      row.names = NULL)
})) |>
pivot_longer(all_of(methods), names_to = "method", values_to = "flag") |>
mutate(method = factor(method, levels = methods),
      flag = ifelse(is.na(flag), "", flag),
      targeting = ifelse(target == 0, "misfit at 0", "misfit at -2"),
      misfit_items = paste(n_misfit, ifelse(n_misfit == 1, "item", "items")))

long |>
distinct(direction, n_misfit, target, n, rep, failed) |>
group_by(direction, n_misfit, target, n) |>
summarise(datasets = n(), failed = first(failed), .groups = "drop") |>
filter(failed > 0)

```

direction	n_misfit	target	n	datasets	failed
overfit	1	-2	150	90	10
overfit	1	-2	250	95	5
overfit	1	-2	400	99	1
overfit	1	0	150	99	1
overfit	2	-2	150	86	14
overfit	2	-2	250	90	10
overfit	2	0	150	99	1
overfit	2	0	250	99	1
underfit	1	-2	150	95	5
underfit	1	-2	250	98	2
underfit	2	-2	150	96	4
underfit	2	-2	250	97	3

Results

Detection of the misfitting items

Proportion of misfitting items flagged in the correct direction.

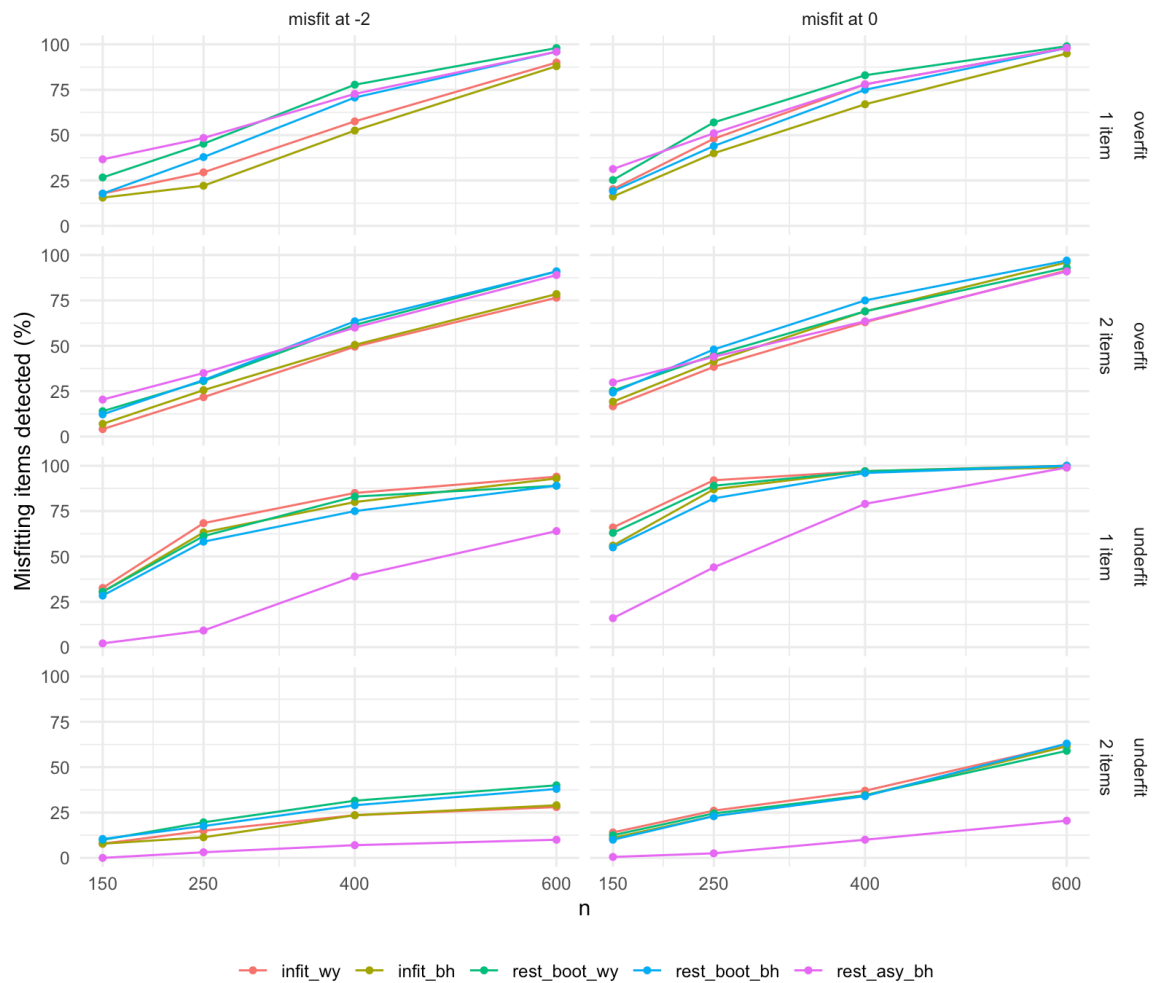
```

det <- long |>
filter(misfit) |>
group_by(direction, misfit_items, targeting, n, method) |>
summarise(correct = 100 * mean(flag == direction),
          wrong = 100 * mean(flag != "" & flag != direction),
          .groups = "drop")

ggplot(det, aes(n, correct, colour = method)) +
  geom_line() +
  geom_point(size = 1.2) +
  facet_grid(direction + misfit_items ~ targeting) +

```

```
scale_x_continuous(breaks = c(150, 250, 400, 600)) +
labs(x = "n", y = "Misfitting items detected (%)", colour = NULL) +
theme_minimal() +
theme(legend.position = "bottom")
```



```
det |>
  select(-wrong) |>
  pivot_wider(names_from = method, values_from = correct) |>
  mutate(across(where(is.double) & !n, \(x) round(x, 0)))
```

direction	misfit_items	targeting	n	infit_wy	infit_bh	rest_boot_wy	rest_boot_bh	rest_asy_bh
overfit	1 item	misfit at -2	150	18	16	27	18	37
overfit	1 item	misfit at -2	250	29	22	45	38	48
overfit	1 item	misfit at -2	400	58	53	78	71	73
overfit	1 item	misfit at -2	600	90	88	98	96	96
overfit	1 item	misfit at 0	150	20	16	25	19	31
overfit	1 item	misfit at 0	250	48	40	57	44	51
overfit	1 item	misfit at 0	400	78	67	83	75	78
overfit	1 item	misfit at 0	600	98	95	99	98	98

direction	misfit_items	targeting	n	infit_wy	infit_bhrest_boot_wy	rest_boot_bh	rest_asy_bh	
overfit	2 items	misfit at -2	150	4	7	14	12	20
overfit	2 items	misfit at -2	250	22	26	31	31	35
overfit	2 items	misfit at -2	400	50	50	62	64	60
overfit	2 items	misfit at -2	600	76	78	91	91	89
overfit	2 items	misfit at 0	150	17	19	25	24	30
overfit	2 items	misfit at 0	250	38	41	45	48	44
overfit	2 items	misfit at 0	400	63	69	69	75	64
overfit	2 items	misfit at 0	600	92	96	93	97	91
under-fit	1 item	misfit at -2	150	33	31	31	28	2
under-fit	1 item	misfit at -2	250	68	63	61	58	9
under-fit	1 item	misfit at -2	400	85	80	83	75	39
under-fit	1 item	misfit at -2	600	94	93	89	89	64
under-fit	1 item	misfit at 0	150	66	56	63	55	16
under-fit	1 item	misfit at 0	250	92	87	89	82	44
under-fit	1 item	misfit at 0	400	97	97	97	96	79
under-fit	1 item	misfit at 0	600	100	99	100	100	99
under-fit	2 items	misfit at -2	150	8	8	10	10	0
under-fit	2 items	misfit at -2	250	15	11	20	18	3
under-fit	2 items	misfit at -2	400	24	24	32	29	7
under-fit	2 items	misfit at -2	600	28	29	40	38	10
under-fit	2 items	misfit at 0	150	14	11	12	10	0
under-fit	2 items	misfit at 0	250	26	23	24	23	2
under-fit	2 items	misfit at 0	400	37	34	34	34	10
under-fit	2 items	misfit at 0	600	62	62	59	63	20

Misfitting items flagged in the wrong direction, all cells combined:

```
det |>
  group_by(direction, method) |>
  summarise(wrong = round(mean(wrong), 1), .groups = "drop") |>
  pivot_wider(names_from = method, values_from = wrong)
```

direction	infit_wy	infit_bh	rest_boot_wy	rest_boot_bh	rest_asy_bh
overfit	0	0	0	0	0
underfit	0	0	0	0	0

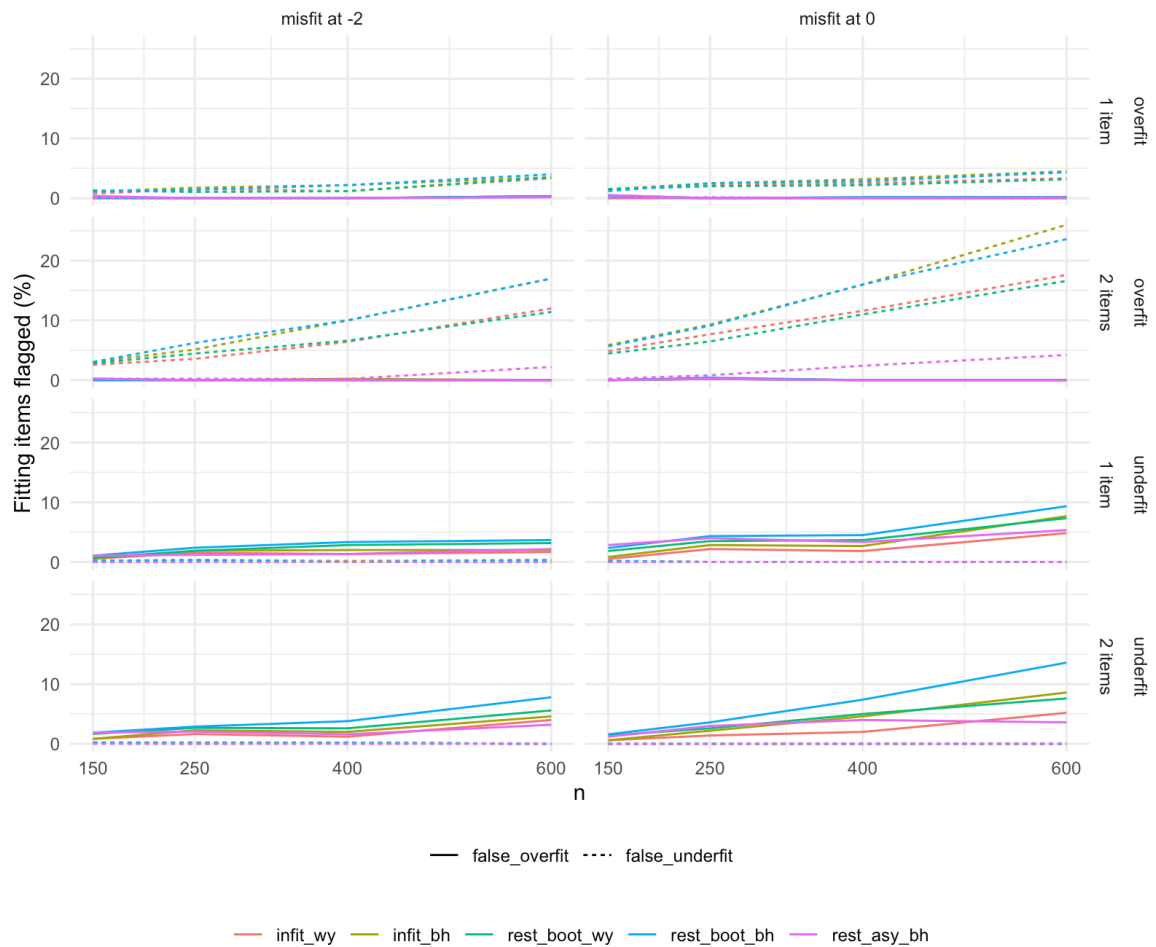
False flags on the fitting items, by direction

This is where the cascade shows. With an underfitting item the fitting items look relatively over-discriminating, so false overfit flags are expected to dominate, and the reverse with an overfitting item.

```
fp <- long |>
  filter(!misfit) |>
  group_by(direction, misfit_items, targeting, n, method) |>
  summarise(false_overfit = 100 * mean(flag == "overfit"),
            false_underfit = 100 * mean(flag == "underfit"),
            .groups = "drop")

fp |>
  pivot_longer(c(false_overfit, false_underfit), names_to = "type",
               values_to = "pct") |>
  ggplot(aes(n, pct, colour = method, linetype = type)) +
  geom_line() +
  facet_grid(direction + misfit_items ~ targeting) +
  scale_x_continuous(breaks = c(150, 250, 400, 600)) +
  labs(x = "n", y = "Fitting items flagged (%)", colour = NULL,
       linetype = NULL,
       title = "Facet rows name the planted misfit") +
  theme_minimal() +
  theme(legend.position = "bottom", legend.box = "vertical")
```

Facet rows name the planted misfit



```
fp |>
  group_by(direction, method) |>
  summarise(false_overfit = round(mean(false_overfit), 1),
            false_underfit = round(mean(false_underfit), 1),
            .groups = "drop")
```

direction	method	false_overfit	false_underfit
overfit	infit_wy	0.0	5.2
overfit	infit_bh	0.1	7.0
overfit	rest_boot_wy	0.1	5.0
overfit	rest_boot_bh	0.1	6.9
overfit	rest_asy_bh	0.1	0.7
underfit	infit_wy	2.0	0.1
underfit	infit_bh	2.9	0.1
underfit	rest_boot_wy	3.4	0.1
underfit	rest_boot_bh	4.6	0.1
underfit	rest_asy_bh	2.6	0.0

Clean decisions

Every misfitting item flagged in the correct direction and no fitting item flagged.

```

long |>
  group_by(direction, misfit_items, targeting, n, method, rep) |>
  summarise(clean = all(flag[misfit] == direction[1]) & all(flag[!misfit] == ""),
            .groups = "drop") |>
  group_by(direction, misfit_items, targeting, n, method) |>
  summarise(clean = round(100 * mean(clean), 0), .groups = "drop") |>
  pivot_wider(names_from = method, values_from = clean)

```

direction	misfit_items	targeting	n	infit_wy	infit_bh	rest_boot_wy	rest_boot_bh	rest_asy_bh
overfit	1 item	misfit at -2	150	18	12	22	13	36
overfit	1 item	misfit at -2	250	25	16	42	34	48
overfit	1 item	misfit at -2	400	52	42	71	61	73
overfit	1 item	misfit at -2	600	74	69	77	72	94
overfit	1 item	misfit at 0	150	18	12	23	15	31
overfit	1 item	misfit at 0	250	37	27	48	34	50
overfit	1 item	misfit at 0	400	66	52	72	62	78
overfit	1 item	misfit at 0	600	80	70	79	73	97
overfit	2 items	misfit at -2	150	0	2	2	2	6
overfit	2 items	misfit at -2	250	4	3	8	7	16
overfit	2 items	misfit at -2	400	12	9	21	18	38
overfit	2 items	misfit at -2	600	22	14	39	20	71
overfit	2 items	misfit at 0	150	1	2	3	2	11
overfit	2 items	misfit at 0	250	4	6	11	7	21
overfit	2 items	misfit at 0	400	11	10	18	14	36
overfit	2 items	misfit at 0	600	25	12	29	16	67
underfit	1 item	misfit at -2	150	33	29	27	23	1
underfit	1 item	misfit at -2	250	61	52	53	44	7
underfit	1 item	misfit at -2	400	77	69	71	59	35
underfit	1 item	misfit at -2	600	85	82	72	67	53
underfit	1 item	misfit at 0	150	63	50	54	41	7
underfit	1 item	misfit at 0	250	79	70	71	59	27
underfit	1 item	misfit at 0	400	86	83	77	72	62
underfit	1 item	misfit at 0	600	71	58	59	53	69

direction	misfit_items	targeting	n	infit_wy	infit_bhrest_boot_wy	rest_boot_bh	rest_asy_bh
underfit	2 items	misfit at -2	150	0	1	0	0
underfit	2 items	misfit at -2	250	0	0	0	0
underfit	2 items	misfit at -2	400	3	3	6	6
underfit	2 items	misfit at -2	600	0	2	1	2
underfit	2 items	misfit at 0	150	1	1	1	1
underfit	2 items	misfit at 0	250	4	3	1	0
underfit	2 items	misfit at 0	400	8	4	5	1
underfit	2 items	misfit at 0	600	27	20	20	9

Paired comparison of restscore and infit

Both tests ran on the same datasets, so the comparison of bootstrap restscore WY with infit WY is paired at the item level. The table counts misfitting items detected by one and not the other, pooled over sample size and the number of misfitting items.

```
long |>
  filter(misfit, method %in% c("infit_wy", "rest_boot_wy")) |>
  mutate(hit = flag == direction) |>
  select(direction, n_misfit, targeting, n, rep, item, method, hit) |>
  pivot_wider(names_from = method, values_from = hit) |>
  group_by(direction, targeting) |>
  summarise(items = n(),
            restscore_only = sum(rest_boot_wy & !infit_wy),
            infit_only = sum(!rest_boot_wy & infit_wy),
            difference_pts = round(100 * (mean(rest_boot_wy) - mean(infit_wy)),
                                   1),
            .groups = "drop")
```

direction	targeting	items	restscore_only	infit_only	difference_pts
overfit	misfit at -2	1136	169	32	12.1
overfit	misfit at 0	1195	78	13	5.4
underfit	misfit at -2	1179	102	65	3.1
underfit	misfit at 0	1200	16	40	-2.0

Failed datasets

A dataset is lost when either cutoff function stops with “All simulation iterations failed”. The check below regenerates every dataset (no bootstrap) and records whether some item has a category observed only among respondents with a zero or perfect total score. CML drops those respondents, so the category is empty in the estimation data, its threshold diverges, and no simulated dataset contains it.

```

extreme_only <- function(d) {
  tot <- rowSums(d)
  inner <- tot > 0 & tot < 4 * ncol(d)
  any(vapply(d, function(v) any(tabulate(v[inner] + 1, 5) == 0), logical(1)))
}
fail_check <- do.call(rbind, lapply(seq_len(nrow(cells)), function(i) {
  x <- readRDS(file.path(RES_DIR, paste0(cells$cell[i], ".rds")))
  failed <- vapply(x$res, is.null, logical(1))
  eo <- vapply(seq_along(x$res), function(r) {
    extreme_only(sim_power_data(
      n = cells$n[i], n_misfit = cells$n_misfit[i], target = cells$target[i],
      direction = cells$direction[i], rho = RHO, slope = SLOPE,
      seed = 1e6 * i + r
    ))
  }, logical(1))
  data.frame(cell = cells$cell[i], failed = sum(failed),
    failed_extreme_only = sum(failed & eo),
    ok_extreme_only = sum(!failed & eo))
}))
fail_check |> filter(failed > 0 | ok_extreme_only > 0)

```

cell	failed	failed_extreme_only	ok_extreme_only
underfit_1item_tm2_n150	5	5	0
underfit_1item_tm2_n250	2	2	0
underfit_2item_tm2_n150	4	4	0
underfit_2item_tm2_n250	3	3	0
overfit_1item_t0_n150	1	1	0
overfit_1item_tm2_n150	10	10	0
overfit_1item_tm2_n250	5	5	0
overfit_1item_tm2_n400	1	1	0
overfit_2item_t0_n150	1	1	0
overfit_2item_t0_n250	1	1	0
overfit_2item_tm2_n150	14	14	2
overfit_2item_tm2_n250	10	10	0

```
colSums(fail_check[, -1])
```

```

      failed failed_extreme_only ok_extreme_only
      57          57              2

```

Interpretation

These are preliminary results at 100 datasets per cell, so differences of a few points between methods within one cell are not reliable. The statements below rest on patterns that repeat across cells or on the pooled paired comparison.

1. Restscore's late underfit flags come from the asymptotic test. The first observation from practice is reproduced by the asymptotic BH rule and by nothing else. With one underfitting item on

target at $n = 150$, infit WY detected 66 percent, bootstrap restscore WY 63 percent and the asymptotic test 16 percent. Pooled over cells, infit was 2 points ahead of bootstrap restscore on target (40 against 16 discordant items) and restscore 3 points ahead off target (102 against 65). The two tests are about equally sensitive to underfit once restscore has a calibrated null.

2. Restscore's extra overfit flags are partly real sensitivity. The bootstrap restscore test detected overfitting items more often than infit, by 5 points on target (78 against 13 discordant items) and by 12 points off target (169 against 32). One overfitting item at -2 logits and $n = 400$ is a typical case, 78 percent against 58. Moving the items off target cost infit more power than it cost restscore. At $n = 150$ the asymptotic test was the most sensitive of the five methods in all four overfit cells, which fits its liberal overfit tail under the null.

3. The cascade produces false flags in the opposite direction, and they grow with n . With two overfitting items planted, the bootstrap methods flagged 8 (WY) to 12 (BH) percent of the fitting items as underfitting, and with underfit planted they flagged 2 to 5 percent as overfitting. Infit and bootstrap restscore behave alike here. The asymptotic test shows almost no underfit cascade (1.3 percent), because its underfit tail hardly ever rejects. That is also why it has the most clean decisions when the planted misfit is overfit (67 and 71 percent with two items at $n = 600$, against 12 to 39 percent for the bootstrap methods). The same conservatism costs it most of its power against underfit, so this is not an argument for the asymptotic test.

4. Two underfitting items mask each other under every method. With two items sharing the second dimension, detection on target rose from 14 percent at $n = 150$ to 62 percent at $n = 600$ (infit WY), and off target to 28 percent (infit WY) and 40 percent (restscore WY). This replicates the masking found in the FWER paper and is a property of the design, not of either test.

5. No misfitting item was flagged in the wrong direction by any method in any cell.

6. Failed datasets follow from CML, not from the methods. 57 of 3,200 datasets were lost, up to 14 of 100 in the off-target cells at $n = 150$. All 57 had an item category observed only among respondents with a zero or perfect total score (as did 2 datasets that ran). All five methods drop the same datasets, so the comparisons stay paired, but the affected cells are conditioned on data the cutoff functions can fit. For users the error message ("All simulation iterations failed. Check your data.") does not say what is wrong, although the cause can be detected before the bootstrap starts.

Limitations. Both cutoff functions used the resample DGP. The validation study found that for restscore it rejects about 1.4 points above nominal under the null, so the two tests are not exactly level-matched. Restscore's advantage against overfit (5 to 12 points) is larger than that, but the smaller differences against underfit are not.